Smart Home Gym: Environmental Monitoring and MediaPipe Exercise Tracking

Salvatore Di Lorenzo
207689

AA 2025/2026

Contents

1 System Architecture **2**
1.1 Hardware and Software Components 2
1.2 Communication Protocols 3
1.3 End-to-End Operational Workflow 3

2 Environmental Node **3**
2.1 Modular Firmware Organization 4
2.2 Sensor Acquisition Strategy 4
2.3 PIR Motion Validation and Vision Wake-Up 4

3 Vision Node **6**
3.1 Deep Sleep Mechanism 6
3.2 MJPEG Streaming over HTTP 6

4 Server-Side Pose Extraction and Exercise Tracking **6**
4.1 Acquisition and Processing Pipeline 7
4.2 Exercise Recognition and Repetition Counting 7

5 Node-RED Orchestration and Sensor Fusion **8**
5.1 Stress Index Estimation 8

6 Performance Discussion and Limitations **9**
6.1 Power Consumption Validation 9
6.2 Streaming Instability and the Abandoned SD-Card Strategy 9
6.3 Limits of Geometry Based Exercise Recognition 10

7 Conclusion and Possible Improvements **10**
7.1 Possible Improvements 10

Introduction

This report presents the design and implementation of a smart home gym system, composed of an environmental node and a vision node. The environmental node is equipped with several sensors providing real-time ambient data, while the vision node features a camera module capable of recording workout videos and streaming them to a local server. The latter analyzes the incoming video stream in real time to identify different types of exercises. Additionally, the local server runs a Node-RED container that acts as the system orchestrator.

1 System Architecture

The end-user interacts with the system through a Node-RED dashboard. This dashboard is populated with data sent from the environment node via MQTT, managed by a centralized broker which Node-RED is subscribed to. Furthermore, Node-RED acts as an MQTT publisher, allowing the user to send specific commands (e.g., start/stop recording) to the Python pose extractor script running on the local server. The script then manages the HTTP connection with the vision node. At the hardware level, the environment node also controls the deep sleep cycle of the vision node via a dedicated wire. Finally, the system leverages a dedicated node to monitor and report the vision node's current consumption.

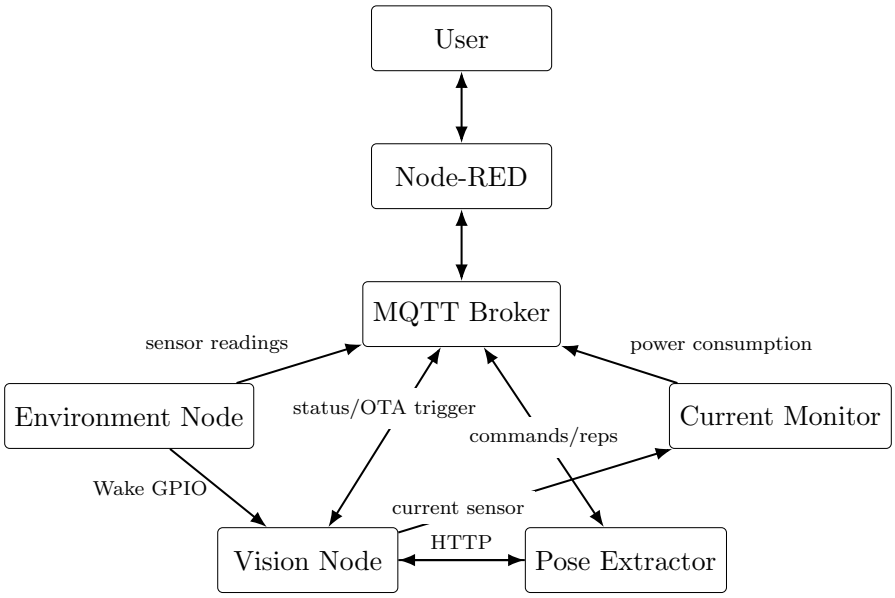

Figure 1: Overall system architecture.

1.1 Hardware and Software Components

The project consists of three microcontroller nodes and one local server. The environmental sensing node is based on an ESP32-S3 and utilizes the following sensors to gather environmental data: an **SHT31** for temperature and humidity, an **MQ7** and an **MQ135** for CO concentration and general air quality, a **PMS5003** for particulate matter concentration, and a **PIR** for motion detection. The environment node also includes a buzzer

and a LED to provide physical feedback depending on the user actions. This node was developed using the ESP-IDF framework.

The vision node, based on an ESP32-CAM, was also developed using ESP-IDF. It provides a video streaming endpoint that is activated only when motion is detected by the environmental node and the appropriate command is sent by the user through Node-RED interface.

The current-monitor node is based on an ESP8266 and measures the current absorbed by the vision node through an INA219 sensor. Since this particular microcontroller is not supported by the ESP-IDF framework, the Arduino framework was used instead.

Finally, the local server hosts the MQTT broker, the Node-RED dashboard, and the Python pose extractor script. The entire system is powered via a wall outlet.

1.2 Communication Protocols

The communication between the system nodes is carried out via MQTT and HTTP over Wi-Fi or Ethernet. The physical layer relies on Wi-Fi for each microcontroller and Ethernet for the local server (which lacks a Wi-Fi antenna).

Regarding the application layer messaging, MQTT was chosen due to its stability and lightness. Both Node-RED and the ESP-IDF framework offer excellent support for this protocol compared to CoAP. Furthermore, since several topics are shared among multiple nodes, using a centralized MQTT broker allows the devices to maintain a single open connection, avoiding the overhead of managing multiple CoAP endpoints over different IPs.

Since the project is designed for short-range communication and does not have strict power-saving constraints, these technologies represent an optimal choice and enable the use of reliable TCP connections.

It is also worth highlighting that the system relies on Quality of Service (QoS) 1 for MQTT messages. The potential reception of duplicate messages is not critical and is gracefully handled by the application logic where necessary (e.g., start and stop commands for the pose extractor script).

1.3 End-to-End Operational Workflow

At a high level, the system operates as follows: the environmental node monitors the garage, validates motion through a software filter, and wakes the vision node only when meaningful motion is confirmed. Once the vision node is active, the recording can be initiated from the dashboard. The Python pose extractor then connects to the camera stream, processes the incoming frames live, publishes tracking results over MQTT, and stores the recordings for later inspection.

2 Environmental Node

This node gathers data from the ambient sensors, manages the PIR sensor implementing a software filter in order to detect actual motion, and handles the wake-up and sleep of the vision node. This node's firmware was implemented aiming for modularity, leaving the `main` module as light as possible.

2.1 Modular Firmware Organization

The node includes dedicated components for Wi-Fi, MQTT, environmental sensors, PMS acquisition, PIR logic, and moving-average filtering. Each component communicates via custom events. This design keeps the `main` module compact and preserves a clean separation between output communication (serial logger and MQTT) and internal application logic. In particular, the `main` module simply performs driver initialization and launches the sensors' reading tasks. When a new event is raised, the `main` module calls the appropriate handler in order to publish an MQTT message, print via serial logger or wake up the vision node.

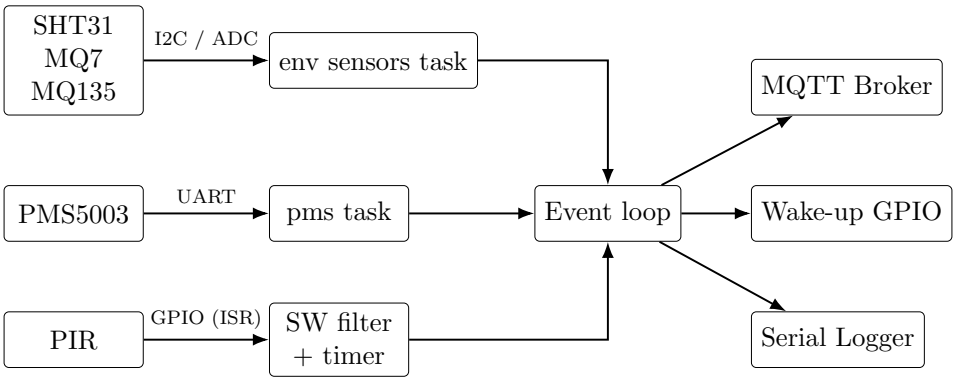

Figure 2: Environment node internal structure.

The node uses distinct communication interfaces to handle each different sensor. The temperature and humidity sensor (SHT31) relies on the standard I2C bus, implemented via a component from the ESP-IDF registry. On the other hand, the PMS5003 particulate matter sensor requires a custom UART decoder in order to parse its 32-byte serial payloads into specific concentration variables. Finally, the gas sensors (MQ7 and MQ135) deliver raw voltage values via the internal ADC. It is noteworthy that getting reliable parts-per-million (ppm) concentration metrics from these sensors requires a proper calibration in clean air.

2.2 Sensor Acquisition Strategy

The firmware spawns two different tasks. The first task reads SHT31, MQ7, and MQ135 sequentially, because these sensors can be polled without requiring a strict acquisition cycle. On the other hand, the second task handles the PMS5003, which must follow a wake–read–sleep cycle with specific timing constraints in order to produce reliable measurements.

Before publication through MQTT, all environmental measurements are filtered with a configurable moving average. The sample size is defined through `menuconfig`, allowing the same firmware to be tuned for faster response or stronger smoothing. In the case of the PMS5003, the firmware also includes a custom UART parser, which validates the packet header and checksum before extracting particulate measurements.

2.3 PIR Motion Validation and Vision Wake-Up

The PIR sensor is particularly sensitive to interference, therefore a single activation would not be a reliable indicator of real movement. For this reason, the environmental node

implements a software filter based on a finite-state machine and on an activation-ratio criterion. Only after sufficient activity has been accumulated within a time window the motion is considered confirmed.

The filter computes the ratio between the total PIR active time $\sum t_{\text{active}}$ and the elapsed time since the first detection:

$$\frac{\sum t_{\text{active}}}{t_{\text{elapsed}}} \geq \frac{\texttt{PIR_ACTIVATION_RATIO_THRESHOLD}}{100} \tag{1}$$

This check is performed only when $t_{\text{elapsed}} \geq$ `PIR_ACTIVATION_TIME`, on both motion detection and motion paused events. If the mathematical condition is met, the motion is confirmed otherwise the filter variables are simply reset.

Once motion is confirmed, the node raises a dedicated GPIO that serves as wake-up source for the vision node. An inactivity timer is then restarted when additional motion confirmations are received. When a time equal to `PIR_TIMEOUT_TIME` expires without new confirmations, the GPIO is set to low and the vision node returns to deep sleep. All the capitalized parameters mentioned above are easily configurable at compile time via `menuconfig`.

Figure 3 illustrates the filter mechanism. Specifically, solid lines represent state transitions triggered by PIR actual signal values, while dashed lines represent state transitions caused by the check on the activation ratio. When the motion is confirmed, a new event is raised and the filter is cleared (see the dotted line). It must be noted that resetting the inactivity timer requires a `MOTION_CONFIRMED` event, rather than just a `MOTION_DETECTED` event.

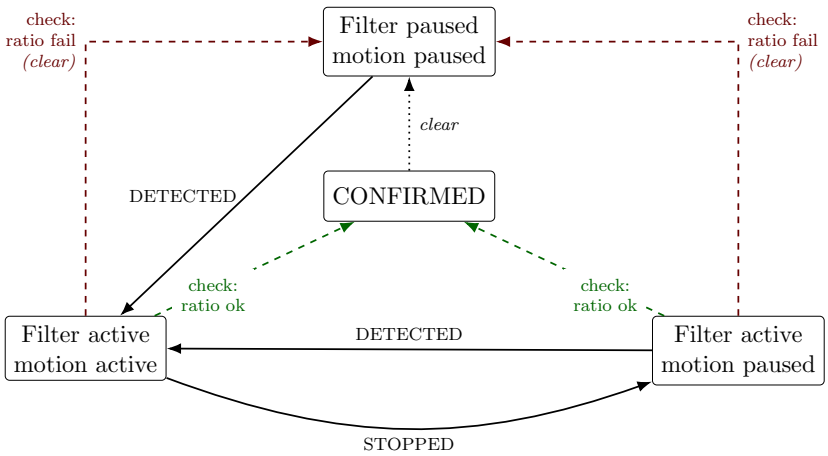

Figure 3: PIR validation finite-state machine.

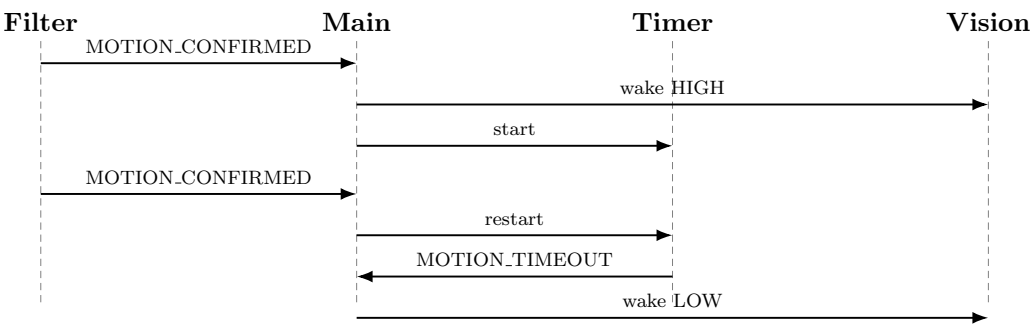

Figure 4: Renewal mechanism used to keep the vision node awake while motion persists.

3 Vision Node

The vision node was designed with a strong focus on energy efficiency. Despite not being needed since the vision node is powered by current, the deep sleep mechanism was implemented in order to conduct a detailed analysis of the power consumption and to experiment with this functionality. The ESP32-CAM remains in deep sleep until the environmental node raises the external wake-up line. This design minimizes idle power consumption and ensures that the camera subsystem is activated only when the gym is actually in use.

3.1 Deep Sleep Mechanism

The firmware accepts wake-up strictly through `EXT0`. If the board boots without a valid GPIO wake-up cause, it immediately returns to deep sleep. Once the node is awake, it initializes networking, MQTT support, camera management, and the streaming server. A dedicated task monitors the wake-up GPIO and returns the device to deep sleep as soon as the signal falls back to zero.

3.2 MJPEG Streaming over HTTP

The vision node exposes an HTTP endpoint, `/stream`, from which the server-side pose extractor script requests a live MJPEG stream. Since the ESP32-CAM is not equipped with a video encoder, the stream is implemented as a sequence of JPEG frames transmitted one after another. In practical terms, the board continuously captures still images and sends them fast enough to emulate video. It should be noted that the camera is active only during streaming, interrupting acquiring new frames upon client disconnection.

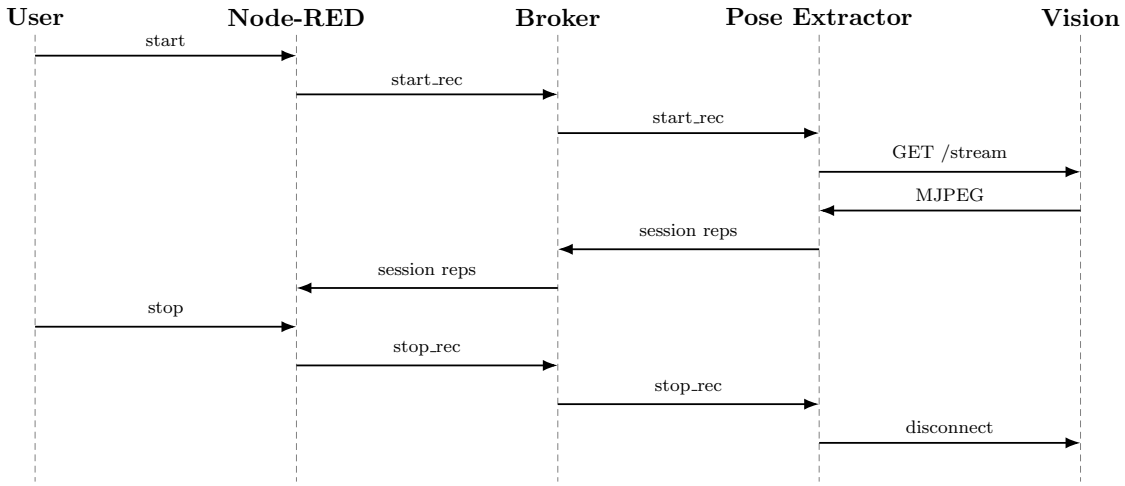

Figure 5: Recording control workflow.

4 Server-Side Pose Extraction and Exercise Tracking

The pose extractor service is the server-side intelligence of the system. It connects to the live MJPEG stream, extracts body landmarks through MediaPipe, and applies a geometry-based tracker to recognize exercises and count repetitions.

4.1 Acquisition and Processing Pipeline

The Python service is activated by MQTT commands and spawns a new tracking session for each recording. A producer thread reads the MJPEG byte stream and extracts JPEG frames, while a consumer thread gets the images, runs MediaPipe, overlays debugging information, and writes the video to disk. This architecture is robust against streaming instability and MediaPipe processing bottlenecks since it operates on a shared buffer between the producer and the consumer. As shown in Figure 6, the final output from MediaPipe is a processed frame that is continuously saved to the video storage, while the script independently sends an MQTT message every time a new repetition is recognized.

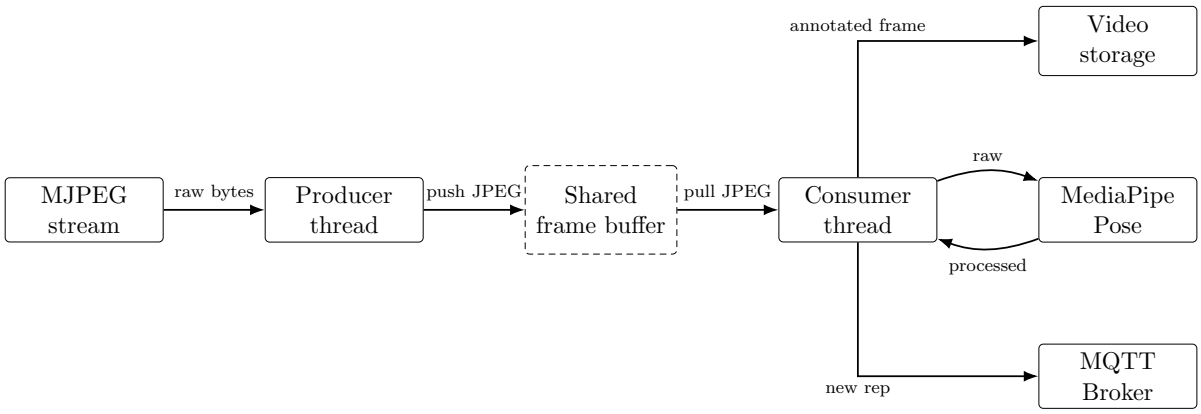

Figure 6: Pose extraction pipeline based on a producer-consumer architecture.

4.2 Exercise Recognition and Repetition Counting

The system uses MediaPipe Pose to estimate body landmarks frame by frame. Only a subset of joints is retained, namely shoulders, elbows, wrists, hips, knees, and ankles. From these points, the tracker derives the geometric angles needed to identify start and end positions for the supported exercises.

The tracking logic is organized as a finite-state machine and is modular. Adding a new exercise simply requires the creation of a new file in the `exercises` module, specifying custom start and end conditions. The current version supports four exercises: Squat, Romanian Deadlift, Lateral Raise, and Bicep Curl.

At the beginning, the tracker is in the *idle* state. When the subject is standing in front of the camera (with both arms along the body), the tracker transitions to the *start-unknown* state. From this point forward, the tracker looks for a possible end position among the predefined end conditions of all supported exercises. Upon detecting a match, the tracker registers the exercise E, having that specific end condition, transitions to the *end-E* state, and waits for the $E_{\text{start_position}}$. When the tracker recognizes the $E_{\text{start_position}}$, it increases the repetition count, returns to the *start-E* state, and monitors for the $E_{\text{end_position}}$, repeating the cycle until the user sends the stop command.

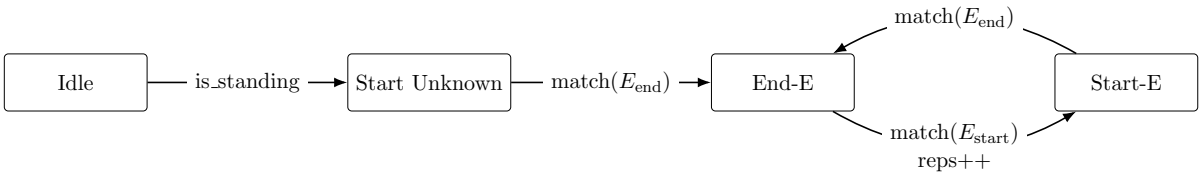

Figure 7: Exercise tracker finite-state machine.

5 Node-RED Orchestration and Sensor Fusion

Node-RED acts as the orchestration layer of the project and exposes the dashboard which allows the end-user to interface with the system. The dashboard is divided into three flows:

- **Environmental flow:** records and displays live ambient data using both gauges and graphs.
- **Vision flow:** provides buttons for sending recording commands and triggering OTA updates, while also displaying a table summarizing the detected exercises. The table is updated in real time for each exercise, since the pose extractor script sends a new MQTT packet for each recorded repetition.
- **Current monitor flow:** provides live current consumption stats for the vision node, visualizing them on a graph.

This arrangement keeps the user interface aligned with the modular organization of the underlying system.

5.1 Stress Index Estimation

The table summarizing the performed exercises also shows a stress index estimation. This index is obtained through a sensor-fusion step, combining environmental conditions and exercise intensity. The first component is the heat index, computed from temperature and relative humidity. In the Node-RED flow, this quantity is estimated through the **NOAA Rothfusz regression**, which is the standard empirical formula used to approximate the apparent temperature perceived by the human body under hot and humid conditions.

This environmental term is then fused with an exercise-dependent MET coefficient and with a custom fatigue multiplier:

$$m_f = 1 + 0.5(\sqrt{r} - 1), \tag{2}$$

where r is the number of repetitions. The reason behind the choice of using the sublinear function is that perceived fatigue tends to increase rapidly at the beginning of a set, at the starting few repetitions, but the fatigue increase should become less aggressive as the repetition count grows. Indeed, the linear multiplier would overestimate the contribution of long sets.

The final stress index is therefore computed as

$$SI = \frac{HI_C \cdot MET_e \cdot m_f}{4.8}, \tag{3}$$

where MET_e is the MET value associated with the recognized exercise. The actual MET value is retrieved based on the recorded activity, ranging from 3.0 to 5.0 as suggested

by several online sources for weightlifting. The denominator was chosen as a practical normalization constant so that a severe but still plausible workout scenario, namely a high heat index, a metabolically demanding exercise such as the squat, and a sufficiently long set (20 reps), corresponds to the upper end of the dashboard scale. The resulting stress index has to be intended as an intuitive way to judge the total exercise fatigue, and it is displayed in the dashboard as an integer value between 0 and 100.

6 Performance Discussion and Limitations

Although the project implements the necessary functionality for the exam requirements, it is not yet suitable for production deployment due to some observed limitations. These issues are discussed in this section, alongside an analysis of the vision node's power consumption.

6.1 Power Consumption Validation

The `current-monitor` node was introduced primarily as a debugging and validation tool. It accurately measures the current absorbed by the vision node and publishes it via MQTT, enabling the Node-RED dashboard to display the real-time trend. This functionality makes the deep-sleep behavior immediately observable. The graph in Figure 8 confirms the expected hardware behavior. Specifically, three stable power levels can be identified: first, a base line (around 2 mA) corresponding to the deep sleep phase; second, an intermediate plateau (around 140 mA) reached when the ESP32 is awake but not actively recording; and lastly, a peak level (around 220 mA) depicting the full-load state, wherein the unit actively captures frames and streams them over Wi-Fi.

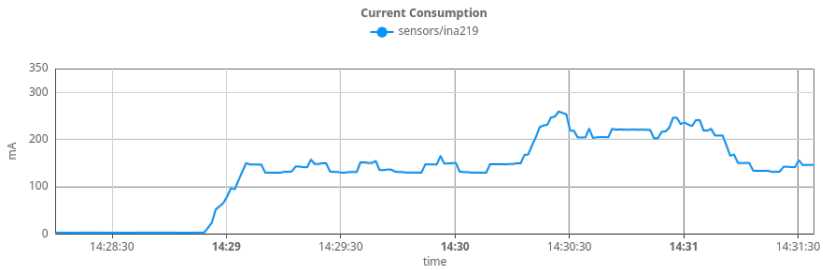

Figure 8: Vision node current consumption captured from the Node-RED dashboard.

6.2 Streaming Instability and the Abandoned SD-Card Strategy

The implemented live-streaming approach is functionally straightforward and aligns with the examples provided by the `esp_camera` component. However, it introduces significant limitations. Specifically, frame acquisition, PSRAM data buffering, and Wi-Fi transmission operations compete for the microcontroller's constrained resources, often resulting in an unstable stream. Indeed, the ESP32-CAM output frequently exhibits instability even when configured for VGA resolution and a relatively low frame rate ($\approx$ 15 fps). This behavior can be attributed also to the limited capabilities of the on-chip Wi-Fi antenna.

An initial design idea was to capture frames, store them locally on an SD card and upload the AVI file only upon completion of the acquisition phase. This would have decoupled video capture from wireless transfer, theoretically improving performance. However,

the practical management of local recording, concurrent requests, and delayed upload would have required a significantly more complex architecture. In addition, simply enabling the local save function would have required porting the `s60sc/ESP32-CAM_MJPEG2S` library (the only suitable option found online) from Arduino to ESP-IDF. Furthermore, SD-card support caused firmware-upload problems due to conflicts on some pins used by both the SD card and the flashing process. This is the reason behind the implementation of OTA update support, which is not strictly required in the implemented live-stream version but remains fully operational.

6.3 Limits of Geometry Based Exercise Recognition

The exercise tracker relies on simple geometric rules and on the detection of start and end poses. While this is sufficient for a small number of exercises, it does not model the temporal dynamics of the full motion. As a result, arbitrary intermediate trajectories may still be counted as valid repetitions, and the recognition logic becomes progressively harder to scale as the number of supported exercises increases. When the tracker is in the *start-unknown* state, it must sequentially evaluate every frame against the end condition of all supported exercises, increasing the compute time. Furthermore, increasing the number of supported exercises raises the probability of overlapping end conditions, leading to the misclassification of exercises.

7 Conclusion and Possible Improvements

The final smart home gym prototype successfully integrates environmental sensing, motion-triggered power management, live video streaming, and real-time server-side pose extraction for exercise tracking, alongside a simple form of sensor fusion.

7.1 Possible Improvements

Since the jittery performance of the vision node live streaming may also be caused by the hardware, a better performance could be achieved by utilizing a separate Wi-Fi antenna or transitioning to a more capable microcontroller. Furthermore, regarding the pose extractor script, a more advanced version could rely on sequence-based machine learning techniques to capture the full movement pattern.